Day 2 Report — Banking Intent Classifier

October 12, 2026

Summary

TF-IDF + Logistic Regression is the strongest model: it reached a Macro-F1 of 0.886 on the test set, about 5 points above the LSTM (0.834).

The goal was to prepare the Banking77 data for fair experiments, then build baseline models and one pipeline that goes from raw text to an intent name. The data has 77 intents, 10,003 examples in the official training file (split into 8,002 train and 2,001 validation), and 3,080 test examples.

Validation and test scores are close. The test set was evaluated once, after the modeling decisions were made, and no model was changed afterwards. The LSTM's validation score is slightly optimistic because early stopping used it, so the test score is the number to rely on.

Experiment Setup (Part 1)

We agreed on a fixed split and a single metric before building any model, so the comparison is fair.

Decision	Choice
Task	Classify a banking customer message into one of 77 intents
Columns	`text` for the message, `category` for the intent
Split	The official train file (10,003) is split 80% train (8,002) and 20% validation (2,001), stratified. The official test file (3,080) was not used for any modeling decision until the final evaluation
Main metric	Macro-F1, because every intent counts equally
Supporting metrics	Accuracy, Weighted-F1, Top-3 accuracy
Seed	42 everywhere
Label encoding	One `LabelEncoder` fitted on train labels, same class order for every model

The dataset has no author, customer, or conversation column, so there are no related examples to group and stratification is appropriate. The smallest intent has 28 training examples and only 7 validation examples, so validation scores for small intents are somewhat noisy. The test set is perfectly balanced (40 examples per intent).

Text Preparation (Track A)

Everything learned from data (vocabulary, maximum length, class labels) is computed from the training texts only, so no data leaks from validation or test.

Preparation steps, in order:

1. Light cleaning: lowercase and collapse whitespace. We keep punctuation and stopwords because they carry meaning (for example "not" and "?").
2. Tokenization into words, with each punctuation mark as its own token. 3. Vocabulary from train only: 2,205 tokens (including PAD and OOV), below the 5,000 cap.
4. Maximum length = 33 tokens (95th percentile of training lengths). The mean length is 13.4, the longest text is 89 tokens, and 4.5% of training texts get truncated. 5. Zero-padding at the end of each text. Matrix shapes: (8,002 × 33) for train, (2,001 × 33) for validation, (3,080 × 33) for test.

The preprocessing is written as a reusable class (`TextPreprocessor`), and the integration point is a single line (`preprocessor = ...`) in cell A2. Automatic checks confirm that matrix shapes, vocabulary indices and labels all match their texts.

Observation on truncation. Truncation cuts the end of long messages. In one inspected example (a long refund complaint of 50 tokens), the cut removed the sentence that states the request ("give me a refund immediately"). Only 4.5% of texts are affected, but it is a possible reason why the LSTM, which sees truncated text, trails TF-IDF, which sees the full text.

Data Leakage Checks

Check	Result
Out-of-vocabulary rate	train 0.00% · validation 0.80% · test 0.71%
Duplicates inside train (after lowercasing)	4
Shared texts, train / validation	0

Check	Result
Shared texts, train / test	7 (0.2% of the test set)

Shared texts, validation / test	0
Near-duplicates of a train text (similarity ≥ 0.80)	validation: 496 of 2,001 · test: 903 of 3,080
Personal information (emails, long digit sequences, URLs)	0

The 7 texts shared between train and test differ only in letter case or spacing, and their effect is negligible, so we kept them and documented them. High similarity with training texts is expected in Banking77, because every intent has many paraphrased wordings.

Models (TrackB)

We built models from simplest to strongest, and recorded the simple scores before training the LSTM.

Dummy (always predicts the most common intent). It always predicts `card_payment_fee_charged` . On validation: Accuracy = 0.019, Macro-F1 = 0.0005. This is the minimum every model must beat.

The near-zero Macro-F1 is expected, not a bug. The Dummy predicts one class out of 77, so only that class gets a non-zero F1 (≈ 0.037) and the other 76 classes get 0. The average is 0.037 / 77 ≈ 0.0005. We also checked the other Dummy strategies on validation:

Strategy	Accuracy	Macro-F1
most_frequent	0.0190	0.0005
stratified	0.0125	0.0117
uniform	0.0155	0.0143

Random strategies land near 1/77 ≈ 0.013. This is also why Macro-F1 is our main metric: accuracy would reward a model that only predicts the largest class.

TF-IDF + Logistic Regression. Uses the same shared `tokenize` function with unigrams and bigrams. On validation: Accuracy = 0.8861, Macro-F1 = 0.8869, Top-3 = 0.968.

Cross-validation (5-fold on the train split): Macro-F1 = 0.871 ± 0.006, with fold scores between 0.866 and 0.882. The tight range shows the result is not a fluke. The Dummy scores 0.0005 ± 0.0000 in the same folds. The vectorizer sits inside the pipeline, so it is

refitted within each fold without leakage.

LSTM. Embedding (100) → LSTM (128) → Dropout (0.3) → Dense (77, softmax), 347,681 parameters in total. A short 3-epoch run proved the pipeline works: training accuracy rose from 7.9% to 52.6% and the loss fell from 3.93 to 1.67.

Integrated Pipeline and FinalEvaluation

We wrapped preparation and model into one object that goes from raw text to an intent name: `TextClassifierPipeline`. If we swap in a better preprocessing step, everything else keeps working unchanged.

The full training used early stopping on validation loss: the best epoch was 14, training stopped at epoch 19, and the weights from epoch 14 were restored. The test set is evaluated once, behind a switch (`RUN_FINAL_TEST`), after all decisions are made.

Validation Results

The LSTM trails TF-IDF by about 6 points of Macro-F1 on validation, a gap similar to the one on the test set.

Model	Accuracy	Macro-F1	Weighted-F1	Top-3
Dummy	0.0190	0.0005	0.0007	0.0490
TF-IDF + Logistic Regression	0.8861	0.8869	0.8863	0.9680
LSTM pipeline	0.8301	0.8296	0.8305	0.9305

Test Set Results

Model	Accuracy	Macro-F1	Weighted-F1	Top-3
Dummy	0.0130	0.0003	0.0003	0.0390
TF-IDF + Logistic Regression	0.8854	0.8858	0.8858	0.9685
LSTM pipeline	0.8344	0.8340	0.8340	0.9481

Answer to the day's question, "is our model better than a simple prediction?": yes, by a wide margin (88.5% accuracy versus 1.3%).

Why the LSTM trailsTF-IDF

It overfits: 97.2% training accuracy versus 82.5% validation accuracy at the last epoch. The dataset is small (8,002 training examples) and the embeddings are learned from scratch.

Its input is truncated at 33 tokens, while TF-IDF sees the full text.

Weakest LSTM intents on validation: `receiving_money` , `supported_cards_and_currencies` , `compromised_card` , `topping_up_by_card` , `top_up_failed` .

Most Confused Intents (validation)

Both models mix up neighbouring intents, such as identity verification, top-ups and pending transfers. Count = number of validation messages.

Model	True intent	Predicted intent	Count
TF-IDF + LogReg	direct_debit_payment_not_recognised	card_payment_not_recognised	6
TF-IDF + LogReg	get_disposable_virtual_card	disposable_card_limits	4
TF-IDF + LogReg	declined_cash_withdrawal	declined_transfer	3
TF-IDF + LogReg	why_verify_identity	unable_to_verify_identity	3
LSTM	transfer_not_received_by_recipient	pending_transfer	5
LSTM	fiat_currency_support	supported_cards_and_currencies	5
LSTM	why_verify_identity	verify_my_identity	5
LSTM	Refund_not_showing_up	request_refund	4
LSTM	top_up_failed	top_up_reverted	4

Predictions on New Sentences (raw text end to end)

Sentence	LSTM	TF-IDF
Why was my transfer declined?	declined_transfer (96.5%)	declined_transfer (88.3%)
How long does a top up take?	pending_top_up (76.2%)	pending_top_up (51.7%)
My card has not arrived yet	card_arrival (92.9%)	card_arrival (44.2%)

Sentence	LSTM	TF-IDF
I lost my card yesterday, please block it	lost_or_stolen_phone (70.4%) — wrong	lost_or_stolen_card (65.6%)

The last row is a sensitive-intent error: the LSTM confuses a lost card with a lost phone, while TF-IDF gets it right.

Next Steps

Treat TF-IDF + Logistic Regression as the main model and the LSTM as an experiment. Improve the LSTM using validation and cross-validation only: more dropout, a bidirectional LSTM, pretrained embeddings, and a longer maximum length (99th percentile) to reduce truncation.

Review the most confused intent pairs on validation for both models.

Day 3 Report — Banking Intent Classifier

October 13, 2026

Summary

TF-IDF + Logistic Regression stays our selected model. It scores Macro-F1 0.887 on validation and 0.886 on test (Day 2), about 5 points above the LSTM.

Day 3 had three goals: improve one model, understand its errors, and prepare the first version of the presentation. What we did:

Experiments (Track B). We ran two LSTM experiments, each changing one setting and scored on validation only.

Error analysis (Track D). We studied the 228 validation errors of the TF-IDF model. In a reviewed sample of 40 errors, 70% came from intents that mean almost the same thing, not from clear model mistakes.

Hardest messages. Short messages (≤ 8 tokens) reach 85.0% accuracy against 90.8% for medium ones. All-lowercase messages reach only 73.6% against 89.3% for the rest.
Confidence. Below a confidence of 0.40, the model is right only 62.8% of the time; above it, 94.2%. The demo therefore says "not sure" for low-confidence messages. **Saved pipeline (Track C).** The model, vocabulary and class names are saved to disk, and a reload check passes.

Prediction notebook. A separate, clean notebook predicts on new text in one cell, in plain words.

Presentation. A slide-by-slide outline is ready, with visuals and placeholders.

Rules for Today

Every Day 3 decision was made on the validation set only. We kept the same experiment setup as Day 2, so the results are directly comparable.

Rule	How we applied it
Same split	8,002 train / 2,001 validation (stratified, seed 42), the same as Day 2
Same tokenizer and labels	The shared `tokenize` function and the shared `LabelEncoder` (77 classes, same order)

Rule	How we applied it
Validation only	The test set was evaluated once on Day 2. Nothing today was tuned or chosen on it
One change at a time	Each experiment changes exactly one setting, so we know what caused the result
At most two experiments	Two LSTM experiments. The Day 2 LSTM stays untouched as the reference
Record every experiment	For each one: what we changed, why we expected it to help, and what happened on validation

Track A — Review ofText Preparation

The Day 2 preparation fits the text well. We changed only the maximum length, because that is the one setting where we found a clear problem.

Check	Finding	Decision
Punctuation and stopwords	Words like "not" and "why", and the "?" mark, separate intents (for example declined vs pending)	Keep them

Lowercasing	Already applied before tokenizing, so letter case cannot confuse the model	Keep
Vocabulary	2,205 tokens, built from training text only. Unseen-word rate: 0.80% on validation	Keep, fitted on train only
Max length (LSTM only)	33 tokens (95th percentile) cuts 4.5% of training texts. The 99th percentile (48 tokens) cuts only 0.9%	Test 48 in Experiment 1
Unseen words	The 177 validation messages with a word never seen in training reach 79.7% accuracy, against 89.5% for the rest. Many are typos or informal words ("topped off", "pop-up")	Note it for the presentation. No change today

Truncation affects only the LSTM. TF-IDF always sees the full message.

The working Day 2 preprocessor was never overwritten. Each experiment builds its own copy, so the original preparation stays available if an experiment does not help.

TrackB — ModelExperiments

We ran two LSTM experiments, each changing one setting. TF-IDF + Logistic Regression (validation Macro-F1 0.8869) remains the bar to beat.

Everything else stayed the same: the architecture (Embedding 100 → LSTM 128 → Dropout → Dense 77), the Adam optimizer (learning rate 0.001), batch size 64, and early stopping on validation loss (patience 5).

Experiment	What we changed	Why we expected it to help	Validation Macro-F1	Validation accuracy
Baseline LSTM (Day 2)	Nothing (reference)	—	0.8296	0.8301

Exp 1: longer sequences	Max length from the 95th to the 99th percentile (33 → 48 tokens)	4.5% of texts were cut. In one example, the cut removed the actual request ("give me a refund immediately")	[fill from notebook]	[fill from notebook]
Exp 2: more regularization	Dropout 0.3 → 0.5 (max length back at 33)	The Day 2 LSTM overfits: 97.2% training vs 82.5% validation accuracy	[fill from notebook]	[fill from notebook]

Decision rule, written before reading the results: we keep an experiment only if it beats the baseline clearly. On 2,001 validation messages, a gain of a few thousandths is noise. We also compare class-level results for the sensitive intents, not only the overall score.

Decision: [fill: kept / not kept, with the difference in Macro-F1]. Either way, the main comparison stays Dummy → TF-IDF + Logistic Regression → LSTM.

We did not try a Transformer or a HuggingFace model today. The guide says to keep the baseline and the LSTM as the main comparison, and two experiments were the limit.

Track D — Error Analysis

Most errors come from intents that mean almost the same thing, not from the model missing the meaning. We started from the working Day 2 TF-IDF model's predictions on validation: 228 errors out of 2,001 messages. All examples below come from the public Banking77 dataset and contain no personal information.

Which labels are confused?

The confused pairs are near-neighbours: payment type, transfer vs balance, and the verification intents. Count = number of validation messages.

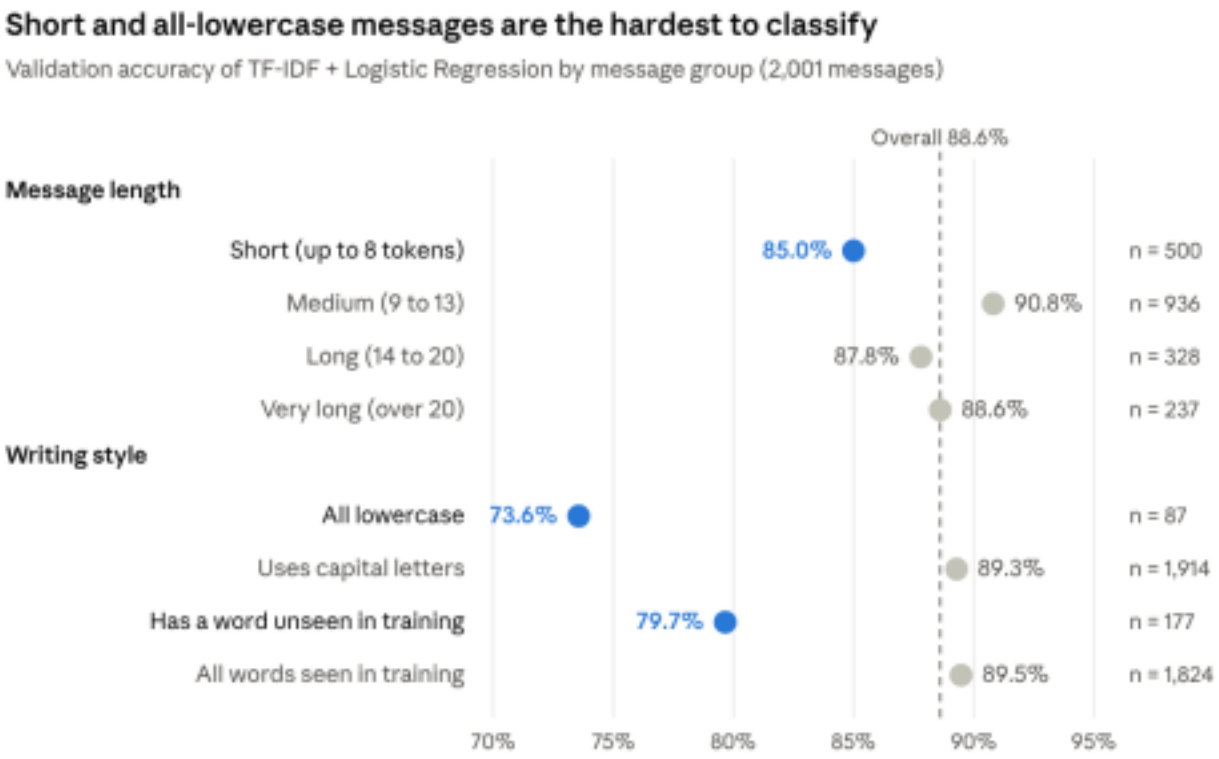
True intent	Predicted intent	Count
direct_debit_payment_not_recognised	card_payment_not_recognised	6
get_disposable_virtual_card	disposable_card_limits	4
declined_cash_withdrawal	declined_transfer	3
why_verify_identity	unable_to_verify_identity	3
change_pin	get_physical_card	3

top_up_failed	pending_top_up	3

The weakest intents (F1) are card_delivery_estimate 0.735, top_up_failed 0.741, declined_card_payment 0.746, pending_transfer 0.754 and lost_or_stolen_card 0.788. The "pending" family is the hardest topic: 76.9% accuracy over 121 messages.

Are short or long texts harder?

Short messages are the hardest. Very short messages carry few clues, and informal writing loses accuracy even more.



Validation predictions of the Day 2 TF-IDF model, re-run on the same split (seed 42), 2,001 messages

All-lowercase messages lose 15 points against the overall accuracy. Messages with a word the model never saw in training lose 9. Lowercasing itself is not the cause, because all text is lowercased before tokenizing; these messages are informal, often with typos and no punctuation.

Is the model reacting to a keyword instead of meaning?

Sometimes. In 20 of the 228 errors, a single word gave at least half of the evidence for the wrong intent:

"PIN" pulls messages toward get_physical_card. "Changing my PIN" and "I cannot use my PIN." were both predicted as get_physical_card, instead of change_pin and pin_blocked.

"debit" pulls toward direct debits. "Is a new debit card free?" was predicted as direct_debit_payment_not_recognised.

"declined" pulls toward declined_transfer, even when the message is about a cash withdrawal.

"deposit" pulls toward cheque or cash deposits, even when the message is about wages

or source of funds.

TF-IDF counts words and does not understand context. This is a known limit of the model, not a data problem.

Model errors or unclear labels?

We read a random sample of 40 errors (seed 42) and sorted each one by hand. This is one reviewer's judgement, so the shares are approximate.

Type of error	Errors (of 40)	Share	Example (text → true label / predicted label)
Ambiguous labels: the text fits both intents	28	70%	"Must I verify my identity?" → why_verify_identity / verify_my_identity
Clear model error: the label is right	10	25%	"There's a possibility that my card was compromised." → compromised_card / declined_card_payment
Label error in the dataset	2	5%	"Can you tell me what i steps i should take since my card was stolen?" → lost_or_stolen_phone / lost_or_stolen_card

Part of the remaining error is therefore a ceiling set by the labels, not the model. On the label errors, the model's answer is actually the correct one.

Does performance differ across groups?

Overall, not across intent types: sensitive intents (lost or stolen card, compromised card, unrecognised payments and similar) reach 89.2%, against 88.5% for the others. Two individual sensitive intents are weaker, though. lost_or_stolen_card has a recall of 0.812 (16 validation messages), and direct_debit_payment_not_recognised has a recall of 0.757.

The gap is in writing style. All-lowercase messages reach 73.6% (87 messages) against 89.3%. Messages with an unseen word reach 79.7% against 89.5%. Customers who write informally, with typos, or in a second language are likely to be served worse. Banking77 is clean, written English, so this gap may be larger with real chat messages.

How confident is the model?

Confidence tells us when to trust the model:

Correct vs wrong. The average confidence is 0.74 for correct predictions and 0.37 for wrong ones.

Below 0.40. 17.7% of validation messages fall below 0.40, and only 62.8% of them are correct. Above 0.40, accuracy is 94.2%.

Top 3. In 71.9% of the errors, the correct intent is still among the model's top 3.

We used the 0.40 threshold, chosen on validation, in the demo. Below it, the message is sent to a human agent instead of being answered automatically.

TF-IDF vs LSTM

The LSTM makes 340 validation errors, against 228 for TF-IDF. Its worst confusions (Day 2) are the same kind of near-neighbours: transfer_not_received_by_recipient → pending_transfer (5) and why_verify_identity → verify_my_identity (5).

Where the two models disagree:

- LSTM wrong, TF-IDF right: [fill].
- TF-IDF wrong, LSTM right: [fill].
- Both wrong: [fill] .

The clearest example is "I lost my card yesterday, please block it". The LSTM predicts lost_or_stolen_phone (70.4%), while TF-IDF correctly predicts lost_or_stolen_card (65.6%).

Examples saved for the presentation

1. **Label error.** "My cards were stolen" is labelled lost_or_stolen_phone in the dataset; our model says lost_or_stolen_card. It shows that the model can be right where the label is wrong.
2. **Keyword error.** "Is a new debit card free?" is predicted as direct_debit_payment_not_recognised because of the single word "debit".

Track C — Saved Pipeline and Prediction Notebook

The full pipeline is saved to disk and reloads with identical predictions. A separate notebook uses it to predict on new text in one cell, with no training.

Midday checkpoint: combine text preparation and model

Checkpoint item	Result
Selected model uses the agreed split, tokenizer and label mapping	Yes: the same 8,002 / 2,001 split, the shared `tokenize` , and the shared `LabelEncoder`
Rerun validation and record the result	TF-IDF + Logistic Regression: accuracy 0.8861, Macro-F1 0.8869. These are identical to Day 2, so the pipeline is reproducible

Checkpoint item	Result

Check confused labels and save privacy safe summaries	Done in Track D. Only counts and public dataset examples are kept
Save the model, tokenizer and label mapping	Saved in the `saved_model/` folder (files below)
Keep an untouched copy	`saved_model.zip` is downloaded and kept unchanged next to the notebook

Saved files

File	Contents	Needed for
`tfidf_logreg.joblib`	The TF-IDF vectorizer and the Logistic Regression in one pipeline	Demo (selected model)
`lstm_model.keras`	The trained LSTM	Comparison model
`vocab.json`	The LSTM vocabulary (index → token) and the maximum length (33)	Turning new text into LSTM input
`classes.json`	The 77 intent names in label encoder order	Turning a predicted number back into an intent name

After saving, both models are reloaded from disk. They give exactly the same predictions as the in-memory models on 200 validation messages (an automatic check in the notebook).

Prediction notebook: `Predict_Intent_Demo.ipynb`

It has three short steps:

1. **Load.** Read the saved model and the class names from `saved_model/`.
2. **Readable answer.** A helper turns the model output into plain words.
3. **Predict.** One clean cell: write a message, run the cell, read the answer.

The notebook defines the same `clean_text` and `tokenize` functions before loading. This is required: the saved TF-IDF pipeline refers to `tokenize` by name, and loading fails without it.

Example output for the demo cell `show_prediction("I lost my card yesterday, please block it")` :

> We predict this customer needs help with: **Lost or stolen card**. Confidence: **66%**. **Urgent**

intent: route this message to the card-security team first. Other possibilities: Lost or stolen phone (6%), Card linking (3%).

If the confidence is below 0.40 (for example, the message "hello" gets 4%), the notebook says it is not sure and suggests passing the message to a human agent. The output never shows a raw array or a class number.

Presentation Outline (First Draft)

The story: customers say the same thing in many ways, our model reads them and routes 89% correctly, and it knows when to ask a human. We build on our existing 5-minute deck and keep its look: rounded cards, one idea per slide, and a short label on every visual.

#	Slide	Main message	Main visual	Status
1	Title	Banking Intent Classifier: from a customer message to the right service	Title slide	Ready (existing deck)
2	The problem	Customers describe the same issue in many ways, and routing them by hand is slow	Three example messages that all mean one intent	Ready (existing "Why does it matter?" slide)
3	The data	13,083 English messages, 77 intents. Training classes are imbalanced, 35 to 187 examples (5.3×). The test set is balanced	Bar chart of examples per intent (Day 1)	Visual saved, slide to build
4	Data surprises	Near-identical messages with different labels, and replies written by the bank inside customer data	Two short example pairs	Placeholder

#	Slide	Main message	Main visual	Status

5	Our approach	Clean lightly and keep punctuation → TF-IDF + Logistic Regression and an LSTM → one shared pipeline	Pipeline diagram: message → tokens → model → intent	Ready (existing "Data & ML" slide, to update)
6	Results	TF-IDF reaches Macro-F1 0.886 on test, against 0.834 for the LSTM and 0.0003 for the Dummy	Bar chart of test Macro-F1 for the three models	Numbers final, chart to make
7	What we tried today	Longer sequences and more dropout for the LSTM: [fill result]	Experiment table (what / why / result)	Placeholder
8	Where the model fails	70% of the sampled errors are intents that mean almost the same. Short and informal messages are harder	Accuracy-by-group chart (Track D) and the "debit" keyword example	Visual ready, slide to build
9	Live demo	Type a message and get the intent in plain words, with an urgent flag and a "not sure" fallback	The prediction notebook, run live	Ready
10	Conclusion and next steps	Simple and explainable wins on this data. Next: merge or redefine look-alike intents, and test on real informal chat	Three short takeaways	Placeholder

Slides that need the most work

- **Slide 4 and slide 8** carry our most original findings. They should get the most care.
- **Slide 6** must show only the test-set scores, so they are not mixed with validation scores.

Privacy

The slides use only short, public Banking77 examples and aggregated numbers. No customer data is shown.

Day 3 Deliverables and Next Steps

All required Day 3 deliverables are in place. The only gaps are the LSTM experiment numbers and the model-disagreement counts, which come from the notebook run.

Required by end of Day 3	Status
A presentation framework with all main slides, visuals, results and conclusions (or placeholders)	Done: 10-slide outline above
A complete working pipeline that runs from top to end	Done: one notebook, from data loading to the saved model
A saved model and tokenizer on disk	Done: `saved_model/` folder plus an untouched zip copy
A clean notebook with headings	Done
A clean prediction notebook that loads the model from disk and predicts in one cell, in human-readable form	Done: `Predict_Intent_Demo.ipynb`

Pitfall	How we avoided it
Changing many things at once	One setting per experiment
Tuning on test examples	Validation only. The test set was not used today
Exposing sensitive examples	Only public dataset examples and counts. No personal data
Chasing a tiny score improvement	A decision rule written in advance, plus class-level checks on sensitive intents
Postponing the slide outline	The outline is drafted today, with visuals assigned

Next steps (Day 4)

- Fill the placeholders: the Exp 1 and Exp 2 results and the disagreement counts.
- Clean the notebook. Its title still says "Day 2", and the models are saved twice (sections 3.6 and 6).
- Push the Day 3 notebook and `Predict_Intent_Demo.ipynb` to GitHub, and fill in the README.
- Build slides 3, 4, 6 and 8 from the saved charts and numbers.
- Rehearse the live demo with 3 or 4 prepared messages, including one "not sure" case.